

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Décrire le modèle de von Neumann et ses 5 unités fonctionnelles
- Expliquer le déroulement du cycle fetch-decode-execute
- Distinguer les différents niveaux de la hiérarchie mémoire (registres, cache, RAM, SSD, HDD)
- Calculer un temps d'accès moyen et comparer les performances
- Relier ces notions aux programmes SNT (architecture) et NSI (performances)

Plan du cours

1. Introduction : qu'est-ce qu'un ordinateur ? (5 min)
2. Le modèle de von Neumann — les 5 unités (20 min)
3. Le cycle fetch-decode-execute (20 min)
4. La hiérarchie mémoire — pyramide des mémoires (20 min)
5. Comparaison des performances et calculs pratiques (15 min)
6. Applications pédagogiques pour SNT/NSI (10 min)

1. Le modèle de von Neumann

```
graph TB
    UC["Unité de Contrôle<br/>(Control Unit)"] --> UAL["UAL<br/>ALU"]
    UC --> MEM["Mémoire<br/>(RAM)"]
    UC --> E_S["Entrées / Sorties<br/>(I/O)"]

    REG["Registres"] --> UAL
    REG --> UC

    MEM --> BUS["Bus de données<br/>Bus d'adresses<br/>Bus de contrôle"]
    BUS --> UC
    BUS --> UAL
    BUS --> E_S

    subgraph "Les 5 unités fonctionnelles"
        U1["1. Unité de Contrôle"]
        U2["2. UAL (Arithmétique & Logique)"]
        U3["3. Mémoire (données + programme)"]
        U4["4. Entrées"]
        U5["5. Sorties"]
    end
    end
```

```
style UC fill:#e74c3c,color:#fff
style UAL fill:#3498db,color:#fff
style MEM fill:#2ecc71,color:#fff
style E_S fill:#f39c12,color:#fff
style REG fill:#9b59b6,color:#fff
style BUS fill:#95a5a6,color:#fff
```

Le modèle de von Neumann (proposé par John von Neumann en 1945) est l'architecture fondamentale de presque tous les ordinateurs modernes. Ses 5 unités sont :

1.1 Unité de Contrôle (Control Unit)

- Dirige et coordonne toutes les actions de l'ordinateur
- Lit les instructions en mémoire et les interprète
- Génère les signaux de contrôle pour les autres unités
- Contient le compteur ordinal (PC — Program Counter) qui pointe vers la prochaine instruction
- Contient le registre d'instruction (IR — Instruction Register) qui stocke l'instruction en cours

1.2 Unité Arithmétique et Logique (UAL / ALU)

- Effectue les opérations arithmétiques : addition, soustraction, multiplication, division
- Effectue les opérations logiques : ET, OU, NON, XOR
- Effectue les comparaisons : égalité, supérieur, inférieur
- Utilise des registres (accumulateurs) pour stocker les opérandes et les résultats

1.3 Mémoire (unité de stockage)

- Stocke à la fois les données et les programmes (= principe de von Neumann)
- Organisée en cellules adressables, chaque cellule contenant un mot mémoire (8, 16, 32 ou 64 bits)
- Deux opérations : lecture (read) et écriture (write)

1.4 Entrées / Sorties (I/O)

- Entrées : clavier, souris, microphone, appareil photo, scanner, réseau
- Sorties : écran, haut-parleurs, imprimante, réseau
- Communication via les bus (données, adresses, contrôle)

1.5 Les bus

- Bus de données : transporte les données entre les unités (bidirectionnel)
- Bus d'adresses : transporte l'adresse mémoire à lire/écrire (unidirectionnel CPU → mémoire)
- Bus de contrôle : transporte les signaux de synchronisation (lecture/écriture, horloge)

Largeur des bus : Un bus d'adresses de 32 bits peut adresser $2^{32} = 4$ Go de mémoire. Un bus de 64 bits peut adresser $2^{64} = 16$ Eo (exaoctets).

2. Le cycle fetch-decode-execute

```
sequenceDiagram
    participant PC as Compteur<br/>Ordinal (PC)
    participant MEM as Mémoire (RAM)
    participant IR as Registre<br/>Instruction
    participant UC as Unité de<br/>Contrôle
    participant UAL as UAL (ALU)
```

Note over PC,UAL: Cycle d'exécution d'une instruction

```
UC->>MEM: 1. FETCH : Envoyer (PC) sur bus d'adresses
MEM-->>UC: Lire instruction à l'adresse PC
UC->>IR: Stocker instruction dans IR
UC->>PC: PC ← PC + 1 (incrémentatation)
```

```
UC->>IR: 2. DECODE : Analyser l'opcode (ex: ADD)
UC->>UC: Identifier les opérandes nécessaires
```

```
UC->>MEM: 3. FETCH Operandes : Lire opérandes
MEM-->>UAL: Transmettre opérandes à l'UAL
```

```
UAL->>UAL: 4. EXECUTE : Effectuer l'opération
UAL-->>MEM: 5. STORE : Écrire le résultat en mémoire
```

Détail des étapes

Étape 1 — Fetch (recherche) :

- L'unité de contrôle place le contenu du PC (compteur ordinal) sur le bus d'adresses
- La mémoire renvoie l'instruction située à cette adresse
- L'instruction est chargée dans le IR (registre d'instruction)
- Le PC est incrémenté pour pointer vers l'instruction suivante

Étape 2 — Decode (décodage) :

- L'unité de contrôle décode l'opcode (code opératoire) pour déterminer l'opération à effectuer
- Elle identifie les opérandes (registres ou adresses mémoire concernés)
- Elle active les circuits appropriés dans l'UAL

Étape 3 — Execute (exécution) :

- L'UAL effectue l'opération (addition, soustraction, comparaison, etc.)
- Le résultat est temporairement stocké dans un registre accumulateur

Étape 4 — Store (rangement) :

- Le résultat est écrit dans un registre ou en mémoire (RAM)
- Le cycle recommence avec l'instruction suivante (pointée par PC)

```
graph TD
    DEBUT("Début") --> FETCH["FETCH<br/>Charger instruction<br/>depuis RAM vers IR"]
    FETCH --> DECODE["DECODE<br/>Analyser l'opcode<br/>dans l'unité de contrôle"]
    DECODE --> EXECUTE["EXECUTE<br/>Effectuer l'opération<br/>dans l'UAL"]
    EXECUTE --> STORE["STORE<br/>Écrire le résultat<br/>en mémoire/registre"]
    STORE --> FETCH
```

```
style DEBUT fill:#2ecc71,color:#fff
style FETCH fill:#3498db,color:#fff
style DECODE fill:#e67e22,color:#fff
style EXECUTE fill:#e74c3c,color:#fff
```

```
style STORE fill:#9b59b6,color:#fff
```

Fréquence d'horloge : Chaque cycle est cadencé par l'horloge du processeur. Un CPU à 3 GHz effectue 3×10^9 cycles par seconde. Une instruction simple (ADD) peut prendre 1 cycle, une instruction complexe (DIV) peut en prendre plusieurs.

3. La hiérarchie mémoire

Pyramide des mémoires

```
graph TB
    subgraph "Hiérarchie mémoire (vitesse décroissante, capacité croissante)"
        R1["Registres<br/>~ 1 Ko<br/>~ 0,3 ns<br/>< 1 cycle"]
        R2["Cache L1<br/>~ 32-64 Ko<br/>~ 1 ns<br/>3-4 cycles"]
        R3["Cache L2<br/>~ 256-512 Ko<br/>~ 3 ns<br/>~ 10 cycles"]
        R4["Cache L3<br/>~ 8-16 Mo<br/>~ 10 ns<br/>~ 30 cycles"]
        R5["RAM (DDR4/5)<br/>~ 8-64 Go<br/>~ 50-100 ns<br/>~ 150 cycles"]
        R6["SSD (NVMe)<br/>~ 256 Go - 2 To<br/>~ 10 000 ns<br/>~ 30 000 cycles"]
        R7["HDD<br/>~ 500 Go - 10 To<br/>~ 10 000 000 ns<br/>~ 30 M cycles"]
    end

    R1 --> R2 --> R3 --> R4 --> R5 --> R6 --> R7

    style R1 fill:#e74c3c,color:#fff
    style R2 fill:#e67e22,color:#fff
    style R3 fill:#f1c40f,color:#000
    style R4 fill:#2ecc71,color:#fff
    style R5 fill:#3498db,color:#fff
    style R6 fill:#9b59b6,color:#fff
    style R7 fill:#34495e,color:#fff
```

Les différentes mémoires

1. Registres :

- Intégrés dans le processeur
- Capacité : quelques dizaines à quelques centaines d'octets
- Temps d'accès : ~ 0,3 ns (1 cycle d'horloge)
- Utilisation : stockage des opérandes et résultats immédiats de l'UAL

2. Mémoire cache (L1, L2, L3) :

- Mémoire rapide et coûteuse, proche du CPU
- Stocke les données et instructions fréquemment utilisées (principe de localité spatiale et temporelle)
- L1 : la plus rapide, dédiée par cœur
- L2 : partagée ou par cœur, moins rapide
- L3 : partagée entre tous les cœurs

3. RAM (Random Access Memory) :

- Mémoire volatile (perdue à l'extinction)
- Capacité : 8 à 64 Go (selon la configuration)
- Temps d'accès : ~ 50 ns (DDR4)

- Organisation : barrettes DIMM/SO-DIMM

4. SSD (Solid State Drive) :

- Mémoire non volatile (Flash NAND)
- Capacité : 256 Go à 2 To
- Temps d'accès : $\sim 10\,000\text{ ns}$ ($10\text{ }\mu\text{s}$)
- Avantages : rapide, silencieux, résistant aux chocs
- Inconvénient : cycles d'écriture limités

5. HDD (Hard Disk Drive) :

- Mémoire magnétique non volatile
- Capacité : 500 Go à 10+ To
- Temps d'accès : $\sim 10\,000\,000\text{ ns}$ (10 ms)
- Avantage : coût par Go très faible
- Inconvénient : lent, mécanique, bruyant

Principe de localité

- Localité temporelle : une donnée accédée récemment sera probablement réutilisée bientôt (boucles)
- Localité spatiale : une donnée proche d'une donnée accédée sera probablement accédée bientôt (tableaux)

Ces principes expliquent l'efficacité du cache : le CPU copie automatiquement en cache les données fréquemment utilisées, ce qui accélère considérablement l'exécution.

Calcul de temps d'accès moyen

Exemple : taux de hit cache = 95 %, temps cache = 1 ns, temps RAM = 50 ns.

$$\text{Temps moyen} = 0,95 \times 1\text{ ns} + 0,05 \times 50\text{ ns} = 0,95 + 2,5 = 3,45\text{ ns}$$

Sans cache, le temps d'accès serait de 50 ns. Avec un cache à 95 % de hits, on gagne un facteur 14.

QCM d'auto-évaluation

- Combien d'unités fonctionnelles composent l'architecture de von Neumann ?
a) 3 unités b) 5 unités ✓ c) 7 unités
- Que signifie l'acronyme UAL ?
a) Unité Arithmétique et Logique ✓ b) Unité d'Application Linéaire c) Unité d'Accès Local
- Dans le cycle fetch-decode-execute, que se passe-t-il à l'étape FETCH ?
a) L'instruction est exécutée par l'UAL b) L'instruction est chargée depuis la RAM ✓ c) Le résultat est stocké en mémoire
- Quelle est la mémoire la plus rapide dans la hiérarchie ?
a) Cache L3 b) RAM c) Registres ✓
- Un SSD est environ combien de fois plus rapide qu'un HDD ?

a) 2 fois b) 100 fois ✓ c) 10 000 fois

Conclusion

L'architecture de von Neumann est le modèle universel de tous les ordinateurs modernes. Les 5 unités (contrôle, UAL, mémoire, entrées, sorties) fonctionnent en synergie via le cycle fetch-decode-execute. La hiérarchie mémoire (registres → cache → RAM → SSD → HDD) est conçue pour concilier vitesse et capacité : plus on s'éloigne du CPU, plus la mémoire est lente et volumineuse. Ces notions sont fondamentales pour comprendre les performances d'un système informatique.

Prochaine séance : Systèmes d'exploitation — processus, threads, ordonnancement et fichiers.

Note pédagogique ENS

Transposition didactique : Pour enseigner l'architecture en SNT Seconde (thème « Architecture machine ») :

- Activité débranchée simuler le cycle fetch-decode-execute avec des élèves jouant les rôles (unité de contrôle, mémoire, UAL)
- Construire une pyramide physique de la mémoire avec des gobelets ou des boîtes pour visualiser la hiérarchie
- Comparer les temps d'accès avec des analogies : registre = votre cerveau (1 sec), cache = bureau (3 sec), RAM = bibliothèque (5 min), SSD = ville voisine (2h), HDD = pays lointain (semaines)

Lien avec les langues : Faire le parallèle entre la mémoire cache (stockage temporaire des données fréquentes) et le vocabulaire actif/passif dans l'apprentissage des langues.

Bibliographie

- von Neumann, J. (1945). First Draft of a Report on the EDVAC. Contrat W-670-ORD-4926.
- Tanenbaum, A. & Austin, T. (2012). Architecture de l'ordinateur (6e éd.). Pearson.
- Hennessy, J. & Patterson, D. (2017). Computer Architecture: A Quantitative Approach (6e éd.). Morgan Kaufmann.
- Goupille, P. (2014). Technologie des ordinateurs. Dunod.
- Dowek, G. et al. (2019). SNT Seconde. Hachette (chapitre « Architecture machine »).